

UNIVERSIDAD DE MONTERREY
Escuela de Ingeniería y Tecnología
Ingeniería en Tecnologías Computacionales

Reporte Técnico

Autor(es)
600046 Fernando Olivares Del Valle

Profesor (a)
Prof. Raúl Morales Salcedo

Doy mi palabra que he realizado esta actividad con integridad académica

San Pedro Garza García, N.L. 27 de Agosto, 2026

Reporte técnico:

1. Descripción del problema y alcance

Para la siguiente actividad se decidió construir una aplicación web monolítica para la gestión de una librería en línea. Dentro de la página se busca gestionar el catálogo de libros, autores, géneros, formatos y conceptos, con control de acceso por rol y con la existencia de un único usuario administrador. El sistema permite el registro de nuevos usuarios, y operaciones CRUD completas sobre todas las entidades del modelo.

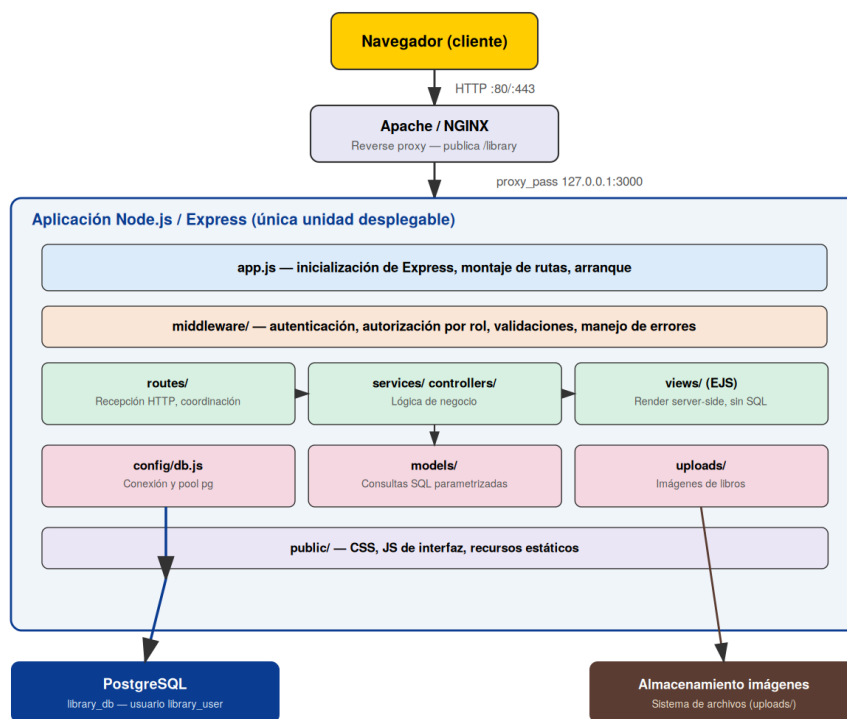
El alcance no incluye la creación de una API o recibir/enviar datos en formato JSON o xML. El servidor arma la página web completa utilizando EJS, renderizando toda la información entre el frontend y backend.

2. Requisitos funcionales y no funcionales

- **Funcionales (RF-01 a RF-12):** registro/login/logout, consulta de catálogo restringida a usuarios autenticados, búsqueda por ISBN y título, CRUD de libros/autores/géneros/ formatos/conceptos, asociación M:N de autores y géneros por libro, conceptos y definiciones por libro, carga/gestión de imágenes con portada única, control de stock y precio, administración restringida al único Administrador, y la regla de negocio de un solo Administrador protegida en base de datos.
- **No funcionales (RNF-01 a RNF-08):** Seguridad (hash de contraseñas, SQL parametrizado, .env fuera de control de versiones), mantenibilidad (separación por capas), integridad de datos (restricciones en PostgreSQL), rendimiento básico, usabilidad, disponibilidad (Node solo en localhost detrás de reverse proxy), trazabilidad de errores, y facilidad de despliegue.

3. Macro-arquitectura y patrón de presentación

Macro-arquitectura Monolítica — WebMonolitico (Librería)



El patrón de presentación utilizado es el Modelo Vista Controlador, adaptado a un server-side con EJS donde se reciben y enrutan las peticiones HTTP por routes, controllers contiene la lógica de negocio, decidiendo qué operaciones realizar. Models se comunica únicamente con PostgreSQL. El middleware se encarga de la autorización por rol y manejo de errores. Views se encarga principalmente de presentar la información al usuario.

4. Organización del código y responsabilidades de los módulos

Módulo	Responsabilidad
app.js	Inicialización de Express, middleware global, montaje de rutas, arranque
src/config/db.js	Configuración y conexión (pool) a PostgreSQL
src/routes/	Recepción de peticiones HTTP y despacho a controladores
src/controllers/	Lógica de negocio y casos de uso
src/models/	Acceso a PostgreSQL con consultas parametrizadas
src/middleware	Autenticación, autorización por rol, manejo de errores
src/views/	Plantillas EJS — solo presentación
public/	CSS, recursos estáticos, imágenes subidas
db/	Scripts SQL: esquema, seed, queries, stored procedures, triggers, vistas

Las tres decisiones arquitectónicas centrales (monolito, acceso directo a PostgreSQL, renderizado server-side con EJS) están justificadas dentro del documento de decisiones ingenieriles.

5. Modelo de datos y normalización hasta 4FN

El modelo tiene 10 tablas: catálogos independientes (formatos, géneros, autores, conceptos), la tabla principal libros, tablas puente para las relaciones que pueden tener varios valores (libro_autor, libro_genero, libro_concepto), la relación de imágenes (donde

cada imagen pertenece a un solo libro), y usuarios. El diagrama ER completo está en docs/DB_DESIGN_ER_4FN.png.

El proceso de normalización (de la forma sin normalizar hasta 4FN) está documentado con ejemplos reales del propio esquema en docs/NORMALIZATION_4FN.pdf. Lo más importante es que autores, géneros, conceptos e imágenes son relaciones independientes entre sí, un libro puede tener varios autores sin que eso tenga que ver con cuántos géneros o imágenes tenga. Por eso cada una se resolvió en su propia tabla puente, para evitar combinaciones que no tienen sentido y así cumplir con la 4FN.

6. Integridad y restricciones de PostgreSQL

Cada tabla tiene sus restricciones justificadas (llave primaria, llave foránea, valores únicos, validaciones y qué pasa al eliminar registros relacionados) en docs/Diseñar integridad y Reglas de negocio en PostgreSQL.pdf. Hay dos reglas de negocio que se protegen directamente en la base de datos, no solo desde el código de la aplicación:

- Un solo administrador: garantiza que como máximo un usuario pueda tener el rol de administrador al mismo tiempo.
- Una sola imagen principal: garantiza que un libro no pueda tener más de una portada marcada.

Se ejecutaron 6 pruebas negativas contra la base de datos real: ISBN duplicado, stock negativo, precio inválido, llave foránea inexistente, eliminación que viola una relación, y creación de un segundo administrador. Todas dieron el resultado esperado, PostgreSQL rechazó la operación en cada caso. El detalle de cada sentencia usada, el resultado esperado y el error real que arrojó Postgres está documentado en docs/Diseñar integridad y Reglas de negocio en PostgreSQL.pdf.

7. Funcionalidades implementadas y flujo navegador

Aquí está un ejemplo del flujo completo con el caso de consultar el catálogo:

1. El navegador pide la página de libros (con sesión activa).
2. El middleware de autenticación verifica que exista una sesión; si no, redirige al login.
3. Las rutas mandan la petición al controlador de libros.
4. El controlador llama al modelo, que ejecuta una consulta parametrizada uniendo las tablas de formatos y autores para armar el catálogo completo.
5. El controlador renderiza la vista de la lista de libros con los datos ya resueltos.
6. El servidor responde con el HTML completo, el navegador solo lo muestra.

Todas las funcionalidades del alcance están implementadas y las probé de forma manual y también mandando peticiones directas al servidor: login, logout, búsqueda por ISBN y título,

CRUD completo de libros, catálogos y usuarios, asociación de varios autores y géneros a un mismo libro, conceptos con su definición por libro, y carga de imágenes con validación.

8. Seguridad

El siguiente es un resumen de los 10 controles que se documentaron dentro de Review de Seguridad.pdf.

1. Hash de contraseñas con bcrypt.
2. Variables de entorno para guardar los secretos, el .env no se sube a git.
3. Consultas SQL parametrizadas al 100%, lo probé intentando un ataque de SQL Injection en el login.
4. Validación del lado del servidor, tanto con restricciones de la base de datos como validaciones en los controladores.
5. Autorización por rol en cada ruta de administración, con un middleware que revisa si el usuario es administrador.
6. Sesiones seguras y un logout que destruye la sesión por completo.
7. Validación de archivos subidos: extensión, tipo de archivo, contenido real del archivo, tamaño máximo de 5MB, y nombre generado por el sistema en vez del nombre original.
8. Manejo de errores centralizado, sin mostrar SQL ni errores internos al usuario.
9. Principio de mínimo privilegio, la aplicación se conecta con un usuario de base de datos sin permisos de superusuario.
10. Ningún secreto (contraseñas, .env, tokens) se publica ni en la entrega ni en el repositorio.

Durante las pruebas encontré algo importante, la validación de imágenes al inicio confiaba en lo que el navegador decía que era el archivo, y eso permitía subir un archivo de texto renombrado como si fuera una imagen. Lo corregí agregando una validación que revisa el contenido real del archivo, no solo su nombre o extensión, y lo volví a probar para confirmar que ya funcionaba bien.

9. Estrategia de pruebas y resultados

Diseñé una matriz de 15 casos de prueba (docs/Plan de Testeo.pdf) que cubre: login y logout, búsqueda, CRUD de libros y catálogos, autorización por rol (visitante, usuario registrado y administrador), la regla de que solo puede haber un administrador, validación de archivos, resistencia a SQL Injection, y la asociación de varias relaciones a un mismo libro. Cada caso incluye precondition, entrada, pasos a seguir, resultado esperado, resultado observado, estado y evidencia.

10. Despliegue mediante Apache/NGINX

Configuré Node.js para que escuche únicamente en 127.0.0.1:3000, sin exponerse directamente a internet. Apache/NGINX funciona como reverse proxy, publicando la aplicación bajo el prefijo /library y reenviando las solicitudes hacia Node. Verifiqué que las rutas estáticas, los formularios, las sesiones y la carga de imágenes funcionaran

correctamente bajo ese prefijo, confirmando el acceso desde un navegador externo al servidor.

11. Limitaciones actuales y riesgos técnicos

Entre las limitaciones actuales y riesgos técnicos encontrados, fueron lo siguientes:

- La validación de los campos depende principalmente de las restricciones de la base de datos, no existe una capa aparte en los controladores que valide cada campo con mensajes específicos.
- No hay límite de intentos de inicio de sesión, así que no hay protección contra ataques de fuerza bruta ni bloqueo temporal después de varios intentos fallidos.
- Las sesiones se manejan en memoria por defecto, así que si en algún momento corren varias instancias de la aplicación al mismo tiempo, las sesiones no se compartirán entre ellas.
- La librería que uso para subir archivos está marcada como obsoleta por sus propios creadores, y tiene vulnerabilidades conocidas en versiones anteriores a la 2.
- No tengo forma de recuperar la contraseña si se me olvida, por ejemplo mandando un correo para restablecerla.
- No existe un mecanismo de recuperación de contraseña.

12. Análisis: evolución hacia componentes desacoplados

Respecto a una posible evolución hacia componentes desacoplados, a un equipo de desarrollo más grande u otros requisitos, estos cambios serían necesarios:

- Separar el acceso a datos en una API REST o GraphQL que devuelva JSON, separando el backend del frontend. Esto rompería la restricción actual del ejercicio de ser un monolito puro sin JSON, pero sería el primer paso natural hacia una arquitectura de servicios.
- Mover el renderizado al lado del cliente, usando algo como React que consuma esa API, para poder actualizar partes de la interfaz sin tener que recargar toda la página.
- Sacar las sesiones a un almacén compartido como Redis, para poder tener varias instancias de la aplicación corriendo al mismo tiempo detrás de un balanceador de carga.
- Guardar las imágenes en un servicio externo en vez de en el disco local, como S3 o su equivalente en GCP, algo necesario si la aplicación llega a correr en varias instancias o contenedores temporales.
- Posiblemente dividir el monolito en servicios separados por dominio, como catálogo, usuarios e imágenes, si el equipo de desarrollo creciera lo suficiente como para justificar el trabajo extra que implican los microservicios: más despliegues independientes, comunicación entre servicios, y más complejidad para monitorear todo.

Estos cambios tienen un costo consciente, se gana escalabilidad y flexibilidad, pero a cambio de más complejidad operativa, más piezas que desplegar y vigilar, y más superficie que proteger en cuanto a seguridad. Son cambios que no se justifican para el alcance

académico de este ejercicio, pero sí tendrían sentido en un entorno de producción con crecimiento real.